RideFlow Beginner Developer Study Notes

Audience: A beginner learning to operate, understand, and safely improve RideFlow
Project: RideFlow private repository
Author: Manus AI
Purpose: Explain the ten essential skill areas needed to understand the RideFlow codebase and gradually become capable of maintaining it.

> You do not need to learn the entire system in one week. Learn one layer at a time, make small changes, run the tests, and ask what happens when something fails. The goal is not to memorize commands; it is to understand how data moves through the application and how to change it safely.

How to use these notes

Read the sections in order. For every topic, run the suggested command, open the named RideFlow file, and complete the exercise before moving on. Keep a notebook containing new words, commands, errors, decisions, and questions. Never use production credentials while learning.

Skill area	Beginner outcome
1. Technology stack	Explain what each language, framework, and tool does
2. Request flow	Trace a button click from browser to server and database
3. Repository reading	Find the correct file before changing code
4. Safe feature changes	Add a small feature without breaking existing behavior
5. Database safety	Change tables and data without accidental loss
6. Debugging	Investigate errors methodically instead of guessing
7. Security	Protect users, secrets, files, and payments
8. Unfinished production work	Distinguish a polished demo from an operational ride service
9. Git and GitHub	Save, review, share, and recover code safely
10. Learning routine	Build skill through deliberate practice

1. Learn the technology stack

What a stack means

A technology stack is the collection of languages, frameworks, libraries, databases, and services used to build an application. RideFlow has a browser layer, a server layer, a database layer, and external service integrations.

Technology	Plain-language meaning	RideFlow location
HTML	Structure of a web page	`client/index.html`, JSX output
CSS	Visual appearance and layout	`client/src/index.css`, Tailwind classes
JavaScript	Browser and server programming language	Throughout the application
TypeScript	JavaScript with static types	`.ts` and `.tsx` files
React	UI library built from components	`client/src/`
Vite	Frontend development and build tool	`vite.config.ts`, `package.json`
Node.js	JavaScript runtime outside the browser	Server process and scripts
Express	HTTP server framework	`server/_core/index.ts`
tRPC	Typed browser-to-server procedure layer	`server/routers.ts`, `client/src/lib/trpc.ts`
Drizzle ORM	TypeScript database mapping/query tool	`drizzle/`, `server/db.ts`
MySQL/TiDB	Relational database	`DATABASE_URL`
S3-compatible storage	Object storage for file bytes	`server/storage.ts`
OIDC/OAuth	Login and identity protocol	`server/_core/oauth.ts`
Vitest	Automated JavaScript/TypeScript testing	`server/*.test.ts`
Git	Version-control system	`.git/` and GitHub

React components describe what the user sees. Node and Express receive requests and run server logic. The database stores durable records. Object storage stores large file bytes. The browser should never receive server-only secrets.

What to learn first

Start with JavaScript variables, functions, arrays, objects, promises, `async/await`, modules, and error handling. Then learn TypeScript types and React state. You do not need advanced mathematics to begin; you need to understand data structures and control flow.

Exercise

Open `client/src/pages/Home.tsx`. Find one component, identify its inputs, state variables, event handlers, and rendered output. Then change one visible label locally, run the development server, and restore the label afterward.

Common beginner mistakes

A common mistake is treating TypeScript as a different language from JavaScript. TypeScript is JavaScript plus a checking layer. Another mistake is editing generated files instead of the source file that generates them. A third mistake is copying a solution without understanding where the data is created or validated.

2. Understand the request flow

When a user clicks a button, several layers may run. RideFlow generally follows this path:

```
User action
  → React event handler
  → tRPC client procedure
  → Express/tRPC server
  → authentication and authorization
  → input validation
  → database/storage/provider operation
  → typed response
  → React loading/success/error state
```

For a fare quote, the browser asks the server to create a quote. The server checks the signed-in user, validates origin/destination and route values, loads fare rules, calculates the KSh total and 5% commission, writes quote/ledger data, and returns the result. The browser displays the response but must not be trusted to determine the payable amount.

How to trace a feature

Choose one feature and search for its visible text or tRPC procedure name. For fare quotes, start in `Home.tsx`, find the quote mutation, locate the matching procedure in `server/routers.ts`, follow the helper in `server/db.ts` or `server/fare.ts`, and then inspect the tables in `drizzle/schema.ts` and tests in `server/fare.test.ts`.

Layer	Question to ask
UI	What did the user click?
Client state	Is the page loading, successful, empty, or in error?
API	Which procedure receives the request?
Authorization	Is the user allowed to perform it?
Validation	Which inputs are checked?
Persistence	Which table or storage bucket changes?
Response	What does the browser display?
Failure	What happens if the database/provider is unavailable?

Exercise

Draw the request flow for profile-photo upload on paper. Label the browser file picker, client mutation, protected procedure, file validation, S3 upload, `rideflow_files` metadata row, and success/error toast.

3. Learn to read the repository

Read the repository from the outside inward. Begin with `README.md`, `package.json`, `client/src/App.tsx`, `client/src/pages/Home.tsx`, `server/routers.ts`, `server/db.ts`, `drizzle/schema.ts`, and `docs/deployment-environment.template`.

Do not open every UI primitive first. Most files under `client/src/components/ui/` are reusable controls. Learn the application flow and data model before customizing them.

Useful search commands

```
# Find a file
find client server drizzle -type f | sort

# Find a word or procedure
rg "fare|quote|admin|presence|activity" client server drizzle

# See recent changes
git log --oneline --decorate -10

# Inspect the current working tree
git status --short
```

Code-reading questions

When reading a function, identify its inputs, return value, side effects, permission requirements, error paths, and dependencies. Ask whether it changes a database row, uploads a file, calls an external provider, or only changes local browser state.

Exercise

Create a one-page map with these entries: frontend entry, main page, tRPC client, server entry, router, database helper, schema, storage adapter, payment adapter, and test file. Write the path and one sentence for each.

4. Make safe feature changes

A safe change is small, testable, reversible, and consistent with existing patterns. Before editing, write the requirement in `todo.md`. Then inspect existing components and procedures that solve a similar problem.

Use this change sequence:

```
Requirement
  → identify affected data and permissions
  → inspect existing component/procedure
  → update schema if durable data is required
  → update database helper
  → update tRPC procedure
  → update UI states
  → add or update tests
  → run check/test/build
  → verify in the browser
  → review diff and commit
```

UI states every feature should consider

A real feature needs loading, success, empty, error, retry, disabled, and unauthorized states. A button that does nothing is not complete. A screen that only shows a success design is not proof that the server works.

Exercise

Add a small non-sensitive UI improvement, such as a clearer empty state for trips. Add a test only if the change affects a procedure. Run:

```
pnpm check
pnpm test
pnpm build
```

Common mistakes

Do not call navigation or state setters during React render. Do not create unstable object inputs for queries on every render. Do not duplicate a component that already exists in the template. Do not make a payment or admin action frontend-only.

5. Learn database safety

A relational database stores structured records in tables. A primary key identifies a row. A unique constraint prevents duplicates. An index helps lookups. A migration changes the database structure in a repeatable, reviewable way. A transaction groups related changes so they succeed or fail together when appropriate.

RideFlow's important tables include `users`, `rideflow_profiles`, `rideflow_files`, `rideflow_admin_settings`, `rideflow_fare_rules`, `rideflow_fare_quotes`, `rideflow_ledger_entries`, `rideflow_presence`, and `rideflow_activity_events`.

Schema-first workflow

```
# edit drizzle/schema.ts
pnpm drizzle-kit generate
# inspect the generated drizzle/*.sql migration
# test against a disposable database
pnpm drizzle-kit migrate
# back up production before applying the approved migration
```

Never run broad `DELETE`, `DROP TABLE`, or destructive updates on production without a verified backup and explicit approval. Financial history should be corrected using refund or reversal entries, not by rewriting history.

Database exercise

Create a disposable database and inspect the tables after migrations. Identify the primary key in each table, the fields that identify ownership, and the fields that should never be exposed to ordinary users.

Data questions to ask

Who owns this row? Can the user read it? Can the user update it? Should it be append-only? Does it contain personal, financial, identity, or location data? What happens if the operation is repeated? How will the data be deleted or retained?

6. Learn debugging methodically

Debugging means reducing uncertainty. Begin with the exact symptom, reproduce it, capture the error, isolate the layer, test one hypothesis, and record the result.

Symptom	First checks
Blank page	Browser console, import path, React render error
Type error	File and line reported by `pnpm check`
Unauthorized response	Login session, cookie, context, protected procedure
Database failure	`DATABASE_URL`, TLS, migrations, permissions, provider status
Upload failure	File type/size, S3 endpoint, bucket permissions, storage key
Payment pending	Callback URL, provider logs, signature, idempotency record
Deploy failure	Build command, start command, Node version, `PORT`, secrets

Log-reading habit

Use timestamps and correlation identifiers. Never print tokens, cookies, passwords, card details, M-Pesa PINs, or raw identity documents. A useful bug report says what action happened, in which environment, at what time, with which non-sensitive ID, and what response occurred.

Exercise

Break a harmless local feature intentionally, such as changing a UI import name. Run `pnpm check`, read the exact error, fix it, and write down the reasoning. Repeat with a test failure.

7. Learn security responsibilities

The browser is controlled by the user. Anything in frontend code can be inspected or modified. Authorization must therefore happen on the server. Hiding an admin button is a usability feature, not a security boundary.

Core security rules

Use HTTPS. Store secrets in a secret manager. Use separate development, sandbox, and production credentials. Enforce authorization in protected/admin procedures. Validate server inputs. Keep object storage private. Use short-lived signed URLs. Verify webhook signatures. Make payment callbacks idempotent. Rate-limit login, uploads, quotes, and payment operations. Log security events without secrets. Enable multi-factor authentication on owner and provider accounts.

Passwords belong to the identity provider. Do not store or hardcode them in RideFlow. If a password has been shared in chat, source code, or a screenshot, rotate it.

Security exercise

For each admin procedure in `server/routers.ts`, answer: What happens if a normal user calls it directly? What happens if the input is malformed? What data is returned? What is logged? Add a regression test if the behavior is not already covered.

Common mistakes

Never trust a client-sent price, role, user ID, payment success flag, or file path. Never put `STRIPE_SECRET_KEY`, database credentials, `JWT_SECRET`, or S3 secret keys in a `VITE_` variable because frontend variables may be exposed to the browser.

8. Understand unfinished production work

A polished interface can make a prototype feel complete. RideFlow still requires several operational systems before a public ride-sharing launch.

Area	What must be built or hardened
Trips	Explicit trip state machine, lifecycle, cancellation, completion, dispute
Dispatch	Driver availability, matching, assignment, reassignment, idempotency
GPS	Authenticated realtime location, privacy, rate limits, reconnect behavior
Safety	Trip sharing, emergency workflow, escalation, incident records
Communication	Customer-driver chat, moderation, retention, abuse handling
Payments	Provider transactions, refunds, disputes, payout reconciliation
Driver operations	Verification review, onboarding workflow, documents, suspension
Customer tools	Scheduled rides, multi-stop trips, favorites, fare splitting
Support	Case management, refunds, lost items, response ownership
Operations	Monitoring, alerts, backups, recovery drills, retention jobs
Compliance	Privacy, transport, payment, employment, and consumer-protection review

A developer should label features honestly as implemented, scaffolded, simulated, or planned. Do not present a simulated nearby-driver count or demo activity event as real operational data.

Exercise

Create a gap matrix with columns for feature, current code location, database requirements, external provider requirements, tests needed, and launch risk. Review it with the owner before building dispatch.

9. Learn Git and GitHub

Git records changes as commits. A branch is an isolated line of work. A pull request is a review conversation. GitHub stores the shared repository. A checkpoint or commit is not a database backup; code and data have separate recovery plans.

Everyday commands

```
# See current changes
git status --short

# Review changes
git diff

# Create a branch
git switch -c feature/clear-trip-empty-state

# Stage selected files
git add client/src/pages/Home.tsx server/routers.ts

# Save a clear commit
git commit -m "Improve trip empty state"

# Share the branch
git push -u origin feature/clear-trip-empty-state

# Review recent history
git log --oneline --decorate -10
```

Review `git diff` before committing. Keep commits small and explain the user-facing or operational reason. Never force-push the main branch without explicit approval. Never use Git to store secrets.

Exercise

Create a branch, make a harmless text change, inspect `git diff`, commit it, and switch back to the main branch. Do not merge it until you understand what changed.

10. Build a practical learning routine

Learn in cycles: read, run, change, test, explain, document. Each study session should have a small outcome. For example, one session can trace a fare quote; the next can add an empty state; the next can inspect the fare tables; the next can add a regression test.

Suggested twelve-week path

Weeks	Focus	Practice outcome
1–2	JavaScript, TypeScript, Git, terminal	Read and make small safe changes
3–4	React, CSS, forms, state	Improve one RideFlow screen with loading/error states
5–6	Node, Express, HTTP, tRPC	Trace and add a small protected procedure
7–8	SQL, MySQL, Drizzle, migrations	Inspect schema and create a disposable migration
9	Authentication and security	Test user/admin authorization and secret handling
10	Storage and payments	Test synthetic upload and sandbox callback behavior
11	Testing and debugging	Add regression tests and diagnose failures
12	Deployment and operations	Deploy a private staging environment and restore a backup

Weekly study template

At the beginning of the week, choose one concept and one RideFlow feature. During the week, read the relevant files, run the application, make one small change, add or update a test, and write a short explanation in your own words. At the end of the week, explain the request flow to another person without looking at the code.

Questions that make you a better developer

Ask: Where does this data come from? Who is allowed to change it? What validates it? Where is it stored? What happens when the provider fails? Can the operation be repeated safely? What information must not be logged? How can the change be tested and rolled back?

Final beginner checklist

Before calling a feature complete, confirm that the code compiles, tests pass, the browser handles loading/empty/error/success states, server authorization is enforced, database changes have migrations, secrets are not exposed, files use private storage, external callbacks are verified, logs are useful but safe, and the documentation explains how to operate and undo the change.

References

[1]: https://react.dev/learn React, “Learn React.”

[2]: https://www.typescriptlang.org/docs/ TypeScript, “Documentation.”

[3]: https://nodejs.org/en/learn Node.js, “Learn Node.js.”

[4]: https://expressjs.com/en/starter/hello-world.html Express, “Hello World example.”

[5]: https://developer.mozilla.org/en-US/docs/Web/HTTP MDN Web Docs, “HTTP.”

[6]: https://git-scm.com/book/en/v2 Git, “Pro Git.”

[7]: https://owasp.org/www-project-top-ten/ OWASP, “OWASP Top 10.”

[8]: https://dev.mysql.com/doc/ MySQL, “MySQL Documentation.”